

How LLM Inference Actually Works

30 concepts, in the exact order they happen from your prompt to the token on screen



AI Coach John



PROFITBRIDGE
Strive For Better Future

Follow

Repost

Everything that happens between **your prompt** and the **answer**

INPUT

1. Prompt
2. System prompt
3. User prompt
4. Tokenization
5. Context window

INFERENCE

6. Prefill phase
7. Attention
8. Positional enc.
9. KV cache
10. Decode phase
11. Logits
12. Softmax

TOKEN SEL.

13. Greedy
14. Temperature
15. Top-k
16. Top-p
17. Beam search
18. Speculative

OUTPUT

19. Max tokens
20. Stop sequences
21. Freq penalty
22. Pres penalty
23. Seed

PERFORMANCE OPTIMIZATION

- | | | |
|-------------------------|----------------------|---------------------------|
| 24. Continuous batching | 25. Dynamic batching | 26. Streaming |
| 27. Prompt caching | 28. Flash Attention | 29. Quantization |
| | | 30. vLLM / PagedAttention |



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

What actually gets sent to the model

- **Prompt:** complete text input sent to the model
- **System prompt:** instructions defining role, behavior, constraints; typically placed first
- **User prompt:** actual request from the person

```
messages = [  
    {"role": "system", "content": "You are a tutor."},  
    {"role": "user", "content": "Explain KV cache."}  
]
```



The model doesn't inherently "know" these are different. They're combined into one token sequence with special markers.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

Text becomes numbers before the model sees anything

The model doesn't read characters or words. It reads **tokens**: word fragments mapped to integer IDs from a fixed vocabulary.

"Tokenization is important"



This is why non-English costs more (more tokens per word) and why models struggle to count letters (they don't see characters).



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

The hard limit on how much the model can see

The maximum number of tokens a model can process in a single request, including **both** your input and its generated output.

CONTEXT WINDOW: 128,000 TOKENS

Sys

Conversation History (120,000)

Remaini



Input and output share the same budget. A 128K context with 120K of input leaves only 8K for generation. When exceeded, earlier details get truncated.



AI Coach John

PROITBRIDGE
Strive For Better Future

Follow

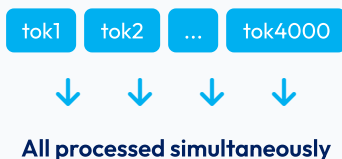
↻ Repost

The first stage of inference:

Prefill Phase

Before generating a single output token, the model processes your entire input prompt in one **parallel** pass, building up internal representations.

PREFILL (Parallel, One Pass)



This phase determines your **Time to First Token (TTFT)**, the latency metric users actually feel.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↺ ↻ Repost

How the model decides what matters, and what came first

- **Attention mechanism:** For each token, computes how much every other token should influence it. Connects "it" to the noun it refers to.
- **Positional encoding:** Attention has no inherent sense of order. Without this, "dog bit man" and "man bit dog" look identical.

"The dog chased the ball because it was fast"

attention weights for "it":

"dog" 0.71

"ball" 0.18

"the" 0.04



Attention is powerful, but its cost grows quadratically with sequence length. This is why long contexts are expensive.



AI Coach John



Follow

↻ Repost

The optimization that makes generation practical

KV Cache: During generation, recomputing attention for every previous token on *every* new token is too slow. KV cache stores those computed keys/values so each new token only computes its own.

Without KV cache:

token 100 → recompute all 99 previous → $O(n^2)$ work

With KV cache:

token 100 → reuse cached 99 → compute only token 100

KV Cache Memory

tok1 [K,V]

tok2 [K,V]

tok3 [K,V]

+ tok4 [K,V] (only this is computed)



KV cache is usually what limits how many concurrent requests a GPU can serve.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Generating output, **one token at a time**

Decode Phase: After prefill, the model generates tokens autoregressively—producing one token, appending it to the sequence, then using that to produce the next.

prefill → token1 → token2 → token3 → [stop]

each token feeds into generating the next

PREFILL: parallel

fast, compute-bound



DECODE: sequential

slow, memory-bound



This is why generating 500 output tokens takes far longer than processing 500 input tokens.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

How the model turns internal state into a token choice

- **Logits:** raw, unnormalized scores for *every* token in the vocabulary.
- **Softmax:** a function converting those raw scores into a proper probability distribution summing to 1.

Logits (raw)

" the"	3.2
" a"	1.1
" cat"	0.4
" zzz"	-2.0

→ Softmax

Probabilities

" the"	0.71
" a"	0.09
" cat"	0.04
" zzz"	0.001



Decoding strategies (temperature, top-k, top-p) are all just different ways of picking a token from these probabilities.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

The two most fundamental generation controls

- **Greedy decoding:** Always pick the single highest-probability token. Deterministic, but repetitive.
- **Temperature:** Reshapes the distribution. Low (0.0-0.3) sharpens it (factual); high (0.8-1.5) flattens it (creative).

temp = 0.2 (sharp)

0.85

0.10

0.03

(picks top almost always)

temp = 1.0 (flat)

0.35

0.28

0.22

(much more variety)



For factual/RAG use cases, keep it low — high temperature is a common cause of unnecessary hallucination.



AI Coach John



Follow

Repost

Two ways to limit which tokens are even considered

- **Top-k sampling:** Only consider the k highest-probability tokens. Fixed count regardless of confidence.
- **Top-p (nucleus):** Consider the smallest set of tokens whose probabilities add up to p (e.g., 0.9). Adaptive.

TOP-K (k=3, fixed)

0.60 ✓

0.25 ✓

0.10 ✓

0.03 ✗

always exactly 3

TOP-P (p=0.9, adaptive)

0.60 ✓

0.25 ✓

0.10 ✓

0.03 ✗ (stop here)

count varies by confidence

} cum = 0.95 ≥ 0.9



Top-p is generally preferred because it adapts to the model's certainty.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

Repost

Making attention **memory-efficient**

Flash Attention: Optimized implementation that avoids writing the huge intermediate attention matrix to slow GPU memory, instead computing it in small tiles in fast on-chip memory.

STANDARD ATTENTION

full $n \times n$ matrix
written to slow HBM



FLASH ATTENTION

computed in tiles
kept in fast SRAM



Mathematically identical results, far less memory, runs faster. This is why long context windows became viable.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↺ ↻ Repost

Shrinking the model to fit and run faster

Quantization: Reducing numerical precision of model weights (e.g., 16-bit to 8-bit or 4-bit) to cut memory usage and increase speed.

FP16 (16-bit)

70B model $\approx$ 140 GB



needs multiple high-end GPUs

INT8 (8-bit)

70B model $\approx$ 70 GB



fits on fewer GPUs

INT4 (4-bit)

70B model $\approx$ 35 GB



runs on single GPU



8-bit is usually nearly lossless. 4-bit noticeably degrades reasoning. This tradeoff is behind Ollama's default quantized downloads.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

Repost

Production-grade inference serving

vLLM & PagedAttention: Manages KV cache memory in fixed-size blocks (like OS virtual memory paging) instead of one large contiguous reservation.

```
python -m vllm.entrypoints.openai.api_server \
  --model mistralai/Mistral-7B-Instruct-v0.2
```

NAIVE KV ALLOCATION

[req A] 

[req B] 

[req C] 

(reserved, unused = waste)

PAGEDATTENTION



blocks, no waste
reused freely



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost

All 30 concepts, in sequence

Prompt (system + user)

↓

Tokenization

→ integer IDs (in context window) ↓

PREFILL:

parallel pass, attention + positional enc, populates KV cache ↳ time to first token ↓

DECODE loop (per token):

logits → softmax → probability distribution ↓ apply: temp, top-k, top-p (or greedy/beam/spec) ↓ selected token → appended, KV cache updated → repeat ↓

STOP when:

max_tokens reached OR stop sequence hit (freq/presence penalties shaped distribution; seed made it reproducible) ↓

Served via:

cont. batching, streaming, prompt caching, Flash Attention, quantization, vLLM/PagedAttention



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

 Repost

Why knowing this makes you better at building with LLMs

Concept understood

→ Better decision made

prefill vs decode

→ cap max_tokens first

TTFT

→ stream + prompt cache

softmax + temperature

→ tune for factuality

seed

→ reliable A/B testing

KV cache memory

→ choose vLLM when self-hosting



Save this for your next AI product build.



AI Coach John



PROITBRIDGE
Strive For Better Future

Follow

↻ Repost